

Sound Sensor

Scripts Breakdown

1. AdvancedSoundSensor

This script acts as the "ears" for the agent, allowing it to detect and process sounds in its vicinity.

Purpose:

- Detect sounds emitted by other objects or agents.
- Track details of the sound (e.g., position, strength, time of detection).
- Provide observations about the most recent sound for decision-making.

Key Components:

- **SoundEvent Struct:** Tracks the details of a single sound event (position, strength, and time heard).
- **Heard Sounds List:** Maintains a list of all recently heard sound events.
- **Memory Decay:** Gradually decreases the strength of heard sounds over time, eventually removing old or weak sounds.

Public Methods:

- `ReceiveSound(Vector3 soundPosition, float baseStrength)`: Registers a sound, calculating its effective strength based on distance and obstacles.
- `Write(ObservationWriter writer)`: Provides the most recent sound data (direction, strength, and position) to the ML-Agents framework.
- `Reset()`: Clears all heard sounds, typically called at the start of an episode.

Key Features:

- **Obstacle Handling:** Reduces sound strength if obstacles are detected between the agent and the sound source.
 - **Sound Memory:** Tracks multiple sounds and decays their strength over time for dynamic decision-making.
-

2. SoundEmitter

This script enables objects or agents to emit sounds into the environment, which can then be detected by nearby **AdvancedSoundSensor** components.

Purpose:

- Broadcast sound events to nearby agents.
- Integrate with specific game actions, such as collisions or footsteps.

Public Methods:

- **EmitSound()**: Emits a sound at the emitter's current position, notifying all nearby agents with sound sensors.

Key Features:

- **Radius of Influence**: Uses a spherical overlap check to determine which agents are within range of the sound.
 - **Customizable Sound Strength**: Allows configuring the base strength of emitted sounds.
-

3. AgentSoccer

This is the main script governing the agents' behavior, integrating both **AdvancedSoundSensor** and **SoundEmitter**.

Purpose:

- Define the agent's role (e.g., Striker or Goalie).
- Enable agents to move, sense, and react to sounds.
- Implement role-specific logic for reacting to sounds.

Key Components:

- **Roles:**
 - **Striker**: Moves toward sound events (e.g., ball collisions).
 - **Goalie**: Moves away from sound events to maintain distance.
- **Thresholds:**
 - **strikerReactionThreshold**: Maximum distance within which the striker reacts to sounds.
 - **goalieReactionThreshold**: Maximum distance within which the goalie reacts to sounds.

Public Methods:

- `Initialize()`: Sets up the agent, defining its team, position, and initial parameters. Initializes the sound sensor and emitter components.
- `MoveAgent(ActionSegment<int> act)`: Processes agent movement based on discrete actions.
- `OnActionReceived(ActionBuffers actionBuffers)`: Handles decision-making, incorporating sound reactions based on the agent's role.
- `OnCollisionEnter(Collision c)`: Emits a sound upon collision with the ball, enabling other agents to react.

Key Features:

- **Integration with Sound System:**
 - Uses `AdvancedSoundSensor` to detect sounds and react accordingly.
 - Emits sounds using `SoundEmitter` during key events, such as ball collisions.
 - **Role-Specific Behavior:**
 - **Strikers**: React to sounds by moving toward them, encouraging proactive behavior.
 - **Goalies**: React to sounds by moving away, maintaining defensive positioning.
-

How the Scripts are Interconnected

1. **Sound Emission:**
 - The `SoundEmitter` component on an agent or object emits sounds during specific events (e.g., ball collisions or movement).
 - The emitted sound's position, strength, and type are broadcast to nearby agents.
 2. **Sound Detection:**
 - Agents equipped with the `AdvancedSoundSensor` detect these sound events within a defined radius.
 - The sensor calculates sound strength based on distance and obstacles and stores it for decision-making.
 3. **Agent Behavior:**
 - The `AgentSoccer` script retrieves the most recent sound data from the sensor and decides how to react based on the agent's role:
 - **Strikers** move toward the sound source if within the reaction threshold.
 - **Goalies** move away from the sound source if within the reaction threshold.
-

Use Case Example

Ball Collision:

- When the ball collides with an object, the ball's **SoundEmitter** emits a sound.
- Nearby agents detect the sound using their **AdvancedSoundSensor**.
- Based on their role:
 - **Strikers** move toward the ball to pursue it.
 - **Goalies** back away to maintain defensive spacing.